Home Assignment: Quantization and Serving Nebius Academy · Performance Engineering This assignment is hands-on. It is a single notebook with two questions. | Question | Topic | Runs on | |----------|-------|---------| | Q1 | Quantization from first principles: implement quantize/dequantize and measure the accuracy vs memory trade-off. | anywhere (CPU is fine) | | Q2 | Serve the same model at BF16 vs on-the-fly 8-bit with vLLM, benchmark both with guidellm, and quantify the memory, throughput, and latency differences. | H100 or L40S | Q1 predicts the win on paper. Q2 measures it on real hardware. ## Notebook `hw_l6_quant_serving.ipynb` Cells are labelled: - 🔒 DO NOT EDIT: the fixed harness (data, plumbing, self-checks, official runs). Editing it invalidates your results. - ✏️ YOUR IMPLEMENTATION: replace `raise NotImplementedError`. These are the only code cells you change. - ✅ SELF-CHECK: fast asserts. These must print `All checks passed` before an official run counts. - ✍️ WRITEUP: answer by editing the markdown cell. Run the notebook top to bottom and keep the outputs. Artifacts land in `results/q1/` and `results/q2/`. ## Hardware Q1 is pure PyTorch on a tiny tensor, so it runs on CPU or any GPU. Q2 needs an NVIDIA GPU. The 8-bit method adapts to the card: - H100 (compute capability 9.0) has native FP8 tensor cores, so it uses `--quantization fp8`. - L40S and other Ada or older cards use `--quantization int8_per_channel_weight_only`. Q2's self-checks run on CPU against a bundled sample report, but your reported Q2 numbers must come from the GPU run. The full run takes about 10 to 15 minutes (two model loads plus four short guidellm benchmarks). ## Getting started ```bash python3 -m venv .venv && source .venv/bin/activate python -m pip install --upgrade pip python -m pip install -r requirements.txt # installs vLLM (brings torch) + guidellm python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))" nvidia-smi jupyter lab # open hw_l6_quant_serving.ipynb ``` Use a fresh virtualenv. vLLM pins its own torch build, and reusing an old environment is the usual cause of CUDA/torch conflicts. ## Choosing a model for Q2 You pick the model to serve. Choose an ungated, dense instruct model on the Hugging Face Hub between about 3B and 8B parameters that vLLM can quantize on the fly, then set it in the model-selection cell near the top of Q2. Avoid gated repos (Llama and Gemma need Hugging Face auth) and mixture-of-experts or unusual architectures, since the on-the-fly quant kernels may not support them. vLLM and guidellm download the model and its tokenizer on first use. Before the full run, smoke-test your pick from a shell so a bad choice fails in seconds instead of mid-benchmark: ```bash vllm serve --quantization fp8 --max-model-len 4096 # use int8_per_channel_weight_only on an L40S ``` Once it prints `server is up`, stop it with Ctrl-C. Your pick is good for the whole assignment. ## Submission Submit: 1. The notebook, executed on your GPU with all outputs visible (both self-checks pass, both official runs printed, all writeups answered). 2. The artifacts under `results/q1/` and `results/q2/` (plots, JSON, and guidellm's per-run `.json` and `.html`). ## A note on integrity You may read the linked docs and library docs. Timings, plots, and benchmark reports are specific to your run, so submit your own.